

10 SECONDS | Science of Success

40 Most Asked Coding Questions for Placements

STORY-BASED PROBLEMS • 10 EASY • 15 MEDIUM • 15 HARD

Each question: the story, how to approach it, full solutions in Python AND Java, complexity, and an extra interview tip — together covering almost the entire DSA syllabus for B.Tech coding rounds.

2,50,000+ learners trained • 55+ college partners

How to use this question bank

- 1. Attempt first.** Read the story and try for 30–40 minutes before opening the approach. Reading solutions without attempting builds recognition, not skill.
- 2. Read the approach before the code.** The "How to approach" bullets are the actual interview answer; the code is only its transcription.
- 3. Re-code from a blank file the next day** in the language of your placement drive. Both Python and Java are given so you can practise either.
- 4. Use the cross-references.** Questions cite each other (e.g., Q39 = Q18 in costume) — following those links is how patterns become permanent.

Note on company tags: company and year attributions are compiled from candidate reports on public interview-prep platforms (LeetCode discussions, GFG experiences, etc.). Treat them as indicative of where each question is popular — companies rotate questions every season.

Question Index — DSA Coverage Map

EASY — Q1-Q10

Q#	Question	DSA topic covered	Asked by (reported)
Q1	The Gift Card Pair (Two Sum)	Arrays · Hash Map	Amazon, Google
Q2	The Code Editor Brackets (Valid Parentheses)	Stack	Microsoft, Amazon
Q3	Reversing the Train (Reverse a Linked List)	Linked List · Pointers	Apple, Amazon
Q4	Merging Two Token Queues (Merge Two Sorted Lists)	Linked List · Two Pointers	Amazon, Oracle
Q5	The Intern's Stock Chart (Best Time to Buy & Sell Stock)	Arrays · Greedy / One Pass	Goldman Sachs, Morgan Stanley
Q6	The Word-Game Referee (Valid Anagram)	Strings · Hashing / Counting	Uber, Amazon
Q7	Finding the Roll Number (Binary Search)	Binary Search	TCS NQT, Infosys, Wipro
Q8	The Best Sales Streak (Maximum Subarray — Kadane's)	Arrays · DP (Kadane's)	Microsoft, LinkedIn
Q9	The Hostel Staircase (Climbing Stairs)	Dynamic Programming (intro)	Adobe, Amazon
Q10	The Missing Attendance Entry (Missing Number)	Bit Manipulation / Math	Microsoft, Amazon

MEDIUM — Q11-Q25

Q#	Question	DSA topic covered	Asked by (reported)
Q11	The Password Screen (Longest Substring Without Repeating Characters)	Sliding Window · Hash Map	Amazon, Adobe
Q12	Three Adjustments That Cancel Out (3Sum)	Sorting · Two Pointers	Adobe, Meta
Q13	Factory Output Without Your Own Machine (Product of Array Except Self)	Arrays · Prefix / Suffix	Meta, Amazon
Q14	The Mirrored Name Tag (Longest Palindromic Substring)	Strings · Expand Around Center	Amazon, Microsoft
Q15	The Vending Machine's Change (Coin Change)	Dynamic Programming	Goldman Sachs, Amazon
Q16	The Growing Salary Records (Longest Increasing Subsequence)	DP · Binary Search	Microsoft, Google
Q17	Counting Islands on the Satellite Map (Number of Islands)	Grid · DFS / BFS	Amazon, Google, Bloomberg
Q18	Ordering the Course Prerequisites (Course Schedule)	Graphs · Topological Sort	Google, Uber
Q19	The Spoiling Crates (Rotting Oranges)	Multi-source BFS	Amazon
Q20	The Trending Hashtags (Top K Frequent Elements)	Hash Map · Heap / Bucket Sort	Meta, Amazon
Q21	Cleaning Up the Meeting Calendar (Merge Intervals)	Sorting · Intervals	Google, Meta
Q22	The Rotated Log File (Search in Rotated Sorted Array)	Modified Binary Search	Amazon, Microsoft
Q23	Auditing the Corrupted Index (Validate Binary Search Tree)	Trees · DFS with Bounds	Amazon, Meta
Q24	Reading the Org Chart Floor by Floor (Binary Tree Level Order Traversal)	Trees · BFS	Amazon, Microsoft
Q25	Every Possible Meal Combo (Subsets)	Backtracking / Recursion	Meta, Amazon

HARD — Q26-Q40

Q#	Question	DSA topic covered	Asked by (reported)
Q26	Water on the City Skyline (Trapping Rain Water)	Arrays · Two Pointers	Amazon, Google
Q27	Median Across Two Sorted Branches (Median of Two Sorted Arrays)	Binary Search · Partitioning	Amazon, Microsoft, Google
Q28	Merging Every Branch's Queue (Merge k Sorted Lists)	Linked Lists · Min-Heap	Amazon, Meta
Q29	The Peak Traffic Monitor (Sliding Window Maximum)	Monotonic Deque	Amazon, Google
Q30	One Letter at a Time (Word Ladder)	Graphs · BFS	Amazon, Meta, Lyft
Q31	The Spell-Checker's Distance (Edit Distance)	Dynamic Programming (2D)	Google, Amazon
Q32	The Boggle Board (Word Search II)	Trie + Backtracking	Amazon, Microsoft
Q33	Seating the Rival Queens (N-Queens)	Backtracking	Amazon, Adobe
Q34	Backing Up the Org Tree (Serialize & Deserialize Binary Tree)	Trees · Design · DFS	Meta, Amazon, LinkedIn
Q35	The Most Profitable Reporting Chain (Binary Tree Maximum Path Sum)	Trees · DFS · DP on Trees	Meta, Amazon
Q36	The Shortest Search Snippet (Minimum Window Substring)	Sliding Window · Hash Counting	Meta, LinkedIn, Amazon
Q37	The Widest Billboard (Largest Rectangle in Histogram)	Monotonic Stack	Amazon, Google
Q38	The Live Median Dashboard (Find Median from Data Stream)	Two Heaps · Design	Amazon, Google
Q39	Decoding the Alien Dictionary (Alien Dictionary)	Graphs · Topological Sort	Google, Airbnb
Q40	The Browser's Memory (LRU Cache)	Design · Hash Map + Doubly Linked List	Amazon, Adobe, Microsoft

DSA coverage check: arrays · strings · hashing · two pointers · sliding window · stack & monotonic stack · queue & deque · linked lists · binary search (3 flavours) · sorting & intervals · heaps (3 uses) · trees & BST · trie · graphs (BFS, DFS, topological sort ×2) · dynamic programming (1D, 2D, on trees) · greedy · backtracking (3 problems) · bit manipulation · design (LRU, median stream, serialization).

EASY

Q1-Q10 · Foundations: arrays, hashing, stack, linked lists, binary search, intro DP, bits. Target: solve each in under 15 minutes.

Q1. The Gift Card Pair (Two Sum)

EASY • Topic: Arrays · Hash Map • Asked by: Amazon, Google (reported 2022–24)

Problem story

Riya is at a store checkout with a gift card of exact value T . The bill screen shows prices of n items. She wants to buy exactly two items whose prices together use up the card completely.

Given the price array and T , return the indices of the two items. Exactly one answer exists; the same item cannot be used twice.

Example

Input : prices = [2, 7, 11, 15], $T = 9$
Output: [0, 1] # $2 + 7 = 9$

How to approach (think before you code)

- **Spot the repeated question.** For each price x , you ask: "has $T - x$ appeared before?" That is a lookup — and fast lookups mean hash map.
- **One pass.** Walk the array; before storing x , check whether its complement is already in the map. This also prevents using the same index twice.
- **Why not sort + two pointers?** Sorting destroys the original indices which the answer needs. Hash map keeps them.

Complexity: Time $O(n)$ · Space $O(n)$

Python

```
def two_sum(prices, T):
    seen = {}                # price -> index
    for i, x in enumerate(prices):
        if T - x in seen:
            return [seen[T - x], i]
        seen[x] = i
```

Java

```
public int[] twoSum(int[] prices, int T) {
    Map<Integer, Integer> seen = new HashMap<>();
    for (int i = 0; i < prices.length; i++) {
        if (seen.containsKey(T - prices[i]))
            return new int[]{ seen.get(T - prices[i]), i };
        seen.put(prices[i], i);
    }
    return new int[]{};
}
```

 **Extra tip:** This exact "seen-before hash map" move upgrades dozens of $O(n^2)$ problems to $O(n)$: pair with difference k , subarray sum equals k , and more. Master it as a pattern, not a problem.

Q2. The Code Editor Brackets (Valid Parentheses)

EASY • Topic: Stack • Asked by: Microsoft, Amazon (reported 2022–24)

Problem story

A code editor must warn programmers when brackets are mismatched. A file contains only the characters `()[]{}.` Return true if every bracket closes the most recently opened bracket of the same type, false otherwise.

Example

Input : `s = "[()]"` → Output: true
 Input : `s = "]"` → Output: false

How to approach (think before you code)

- **Nesting = most-recent-first.** The bracket that must close next is the one opened last. "Last in, first out" is the definition of a stack.
- **Push opens, match closes.** On an opening bracket, push it. On a closing bracket, the stack top must be its partner — pop and compare; any mismatch or empty stack means invalid.
- **End condition.** A valid string leaves the stack empty. Leftover opens (like `"("`) are also invalid — beginners forget this.

Complexity: Time $O(n)$ · Space $O(n)$

Python

```
def is_valid(s):
    pair = {"(": ")", "[": "]", "{": "}"
    st = []
    for ch in s:
        if ch in "([{":
            st.append(ch)
        elif not st or st.pop() != pair[ch]:
            return False
    return not st
```

Java

```
public boolean isValid(String s) {
    Deque<Character> st = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(' || c == '[' || c == '{') st.push(c);
        else {
            if (st.isEmpty()) return false;
            char open = st.pop();
            if ((c == ')' && open != '(') ||
                (c == ']' && open != '[') ||
                (c == '}' && open != '{')) return false;
        }
    }
    return st.isEmpty();
}
```

 **Extra tip:** Test the three killer inputs before submitting: `""` (empty → true), `"("` (leftover open → false), `"]"` (close with empty stack → false).

Q3. Reversing the Train (Reverse a Linked List)

EASY • Topic: Linked List • Pointers • Asked by: Apple, Amazon (reported 2022–24) **Problem story**

A goods train is modelled as a singly linked list of coaches; each coach only knows the coach behind it. The yard master wants the entire train reversed — without building a new train ($O(1)$ extra space).

Return the new head after reversing the links in place.

Example

Input : 1 → 2 → 3 → 4 → null

Output: 4 → 3 → 2 → 1 → null

How to approach (think before you code)

- **Three pointers.** Keep prev (reversed part), curr (node being flipped), and save curr.next before you overwrite it — otherwise the rest of the list is lost.
- **One flip per step.** Point curr.next back to prev, then slide both pointers forward. When curr becomes null, prev is the new head.
- **Trace it once on paper.** Draw 3 nodes and simulate — pointer bugs disappear when you can see the arrows.

Complexity: Time $O(n)$ · Space $O(1)$

Python

```
def reverse_list(head):
    prev = None
    curr = head
    while curr:
        nxt = curr.next      # save before breaking the link
        curr.next = prev
        prev = curr
        curr = nxt
    return prev
```

Java

```
public ListNode reverseList(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode nxt = curr.next;
        curr.next = prev;
        prev = curr;
        curr = nxt;
    }
    return prev;
}
```

 **Extra tip:** The follow-up is almost always "now do it recursively" — practise both. Recursive: reverse the tail, then head.next.next = head; head.next = null.

Q4. Merging Two Token Queues (Merge Two Sorted Lists)

EASY • Topic: Linked List • Two Pointers • Asked by: Amazon, Oracle (reported 2022–24)

 Problem story

Two bank counters each hold a sorted linked list of waiting tokens. The counters are merging — combine both lists into one sorted list, reusing the existing nodes.

Example

Input : 1 → 2 → 4 and 1 → 3 → 4

Output: 1 → 1 → 2 → 3 → 4 → 4

How to approach (think before you code)

- **Dummy head trick.** Start with a dummy node so you never special-case "which list gives the first node?". Return dummy.next at the end.
- **Pick the smaller front.** Compare the two current heads, attach the smaller to the tail, advance that list. When one list ends, attach the other in one step.
- **This IS merge sort's merge.** Learn it well — it reappears in Merge k Sorted Lists (Q28) and external sorting questions.

Complexity: Time $O(m+n)$ · Space $O(1)$

Python

```
def merge(l1, l2):
    dummy = tail = ListNode(0)
    while l1 and l2:
        if l1.val <= l2.val:
            tail.next, l1 = l1, l1.next
        else:
            tail.next, l2 = l2, l2.next
        tail = tail.next
    tail.next = l1 or l2
    return dummy.next
```

Java

```
public ListNode merge(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0), tail = dummy;
    while (l1 != null && l2 != null) {
        if (l1.val <= l2.val) { tail.next = l1; l1 = l1.next; }
        else { tail.next = l2; l2 = l2.next; }
        tail = tail.next;
    }
    tail.next = (l1 != null) ? l1 : l2;
    return dummy.next;
}
```

 **Extra tip:** The dummy-node trick removes head-handling edge cases in almost every linked-list build problem. Reach for it by default.

Q5. The Intern's Stock Chart (Best Time to Buy & Sell Stock)

EASY · Topic: Arrays · Greedy / One Pass · Asked by: Goldman Sachs, Morgan Stanley (reported 2022–24)

Problem story

An intern is shown a stock's price for n consecutive days. She may buy on one day and sell on a single later day. Return the maximum profit possible; if no profit is possible, return 0.

Example

Input : prices = [7, 1, 5, 3, 6, 4]

Output: 6 # buy at 1, sell at 6

How to approach (think before you code)

- **Reframe.** Best profit selling today = today's price – cheapest price seen so far. So carry one running value: the minimum so far.
- **One pass, two variables.** Update min_so_far and best profit at each day. No nested loops needed.
- **Why brute force fails at scale.** n up to $10^5 \rightarrow O(n^2)$ pair checking is 10^{10} operations. The constraint itself demands $O(n)$.

Complexity: Time $O(n)$ · Space $O(1)$

Python

```
def max_profit(prices):
    min_so_far = float("inf")
    best = 0
    for p in prices:
        min_so_far = min(min_so_far, p)
        best = max(best, p - min_so_far)
    return best
```

Java

```
public int maxProfit(int[] prices) {
    int minSoFar = Integer.MAX_VALUE, best = 0;
    for (int p : prices) {
        minSoFar = Math.min(minSoFar, p);
        best = Math.max(best, p - minSoFar);
    }
    return best;
}
```

 **Extra tip:** Strictly falling prices must return 0, not a negative number — interviewers test [5,4,3,2,1] deliberately.

Q6. The Word-Game Referee (Valid Anagram)

EASY · Topic: Strings · Hashing / Counting · Asked by: Uber, Amazon (reported 2022–24)

Problem story

In a word game, a player claims their word t is a rearrangement of the challenge word s . Build the referee: return true if t uses exactly the same letters as s with the same counts.

Example

Input : $s = \text{"listen"}, t = \text{"silent"} \rightarrow \text{Output: true}$
 Input : $s = \text{"rat"}, t = \text{"car"} \rightarrow \text{Output: false}$

How to approach (think before you code)

- **Anagram = same frequency table.** Count letters of s up, letters of t down in one array of size 26. All zeros at the end $\Leftrightarrow$ anagram.
- **Fast reject.** Different lengths can never be anagrams — check that first, it saves the whole scan.
- **Sorting also works** (sort both, compare) but costs $O(n \log n)$; counting is $O(n)$ and shows stronger fundamentals.

Complexity: Time $O(n)$ · Space $O(1)$ (26 counters)

Python

```
def is_anagram(s, t):
    if len(s) != len(t):
        return False
    count = [0] * 26
    for a, b in zip(s, t):
        count[ord(a) - 97] += 1
        count[ord(b) - 97] -= 1
    return all(c == 0 for c in count)
```

Java

```
public boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false;
    int[] count = new int[26];
    for (int i = 0; i < s.length(); i++) {
        count[s.charAt(i) - 'a']++;
        count[t.charAt(i) - 'a']--;
    }
    for (int c : count) if (c != 0) return false;
    return true;
}
```

 **Extra tip:** If the follow-up allows Unicode, switch the fixed array to a HashMap — say this proactively; it shows you read constraints.

Q7. Finding the Roll Number (Binary Search)

EASY • Topic: Binary Search • Asked by: TCS NQT, Infosys, Wipro (reported 2022–24)

Problem story

A college register lists student roll numbers in sorted order. Given a target roll number, return its index in the register, or -1 if the student is absent — fast enough for a register of a million entries.

Example

Input : nums = [-1, 0, 3, 5, 9, 12], target = 9
Output: 4

How to approach (think before you code)

- **Halve the search space.** Compare the middle element with the target; discard the half that cannot contain it. A million entries need only ~20 comparisons.
- **Get the template exact.** $lo \leq hi$, $mid = lo + (hi - lo) / 2$ (overflow-safe in Java), move $lo = mid + 1$ or $hi = mid - 1$. Off-by-one here causes infinite loops.
- **This template is a building block** for rotated arrays (Q22), binary search on answer (playbook §3.2), and Median of Two Sorted Arrays (Q27).

Complexity: Time $O(\log n)$ • Space $O(1)$

Python

```
def search(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target:
            return mid
```

```

    if nums[mid] < target:
        lo = mid + 1
    else:
        hi = mid - 1
return -1

```

Java

```

public int search(int[] nums, int target) {
    int lo = 0, hi = nums.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2; // avoids int overflow
        if (nums[mid] == target) return mid;
        if (nums[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}

```

 **Extra tip:** In Java/C++, always write $lo + (hi - lo) / 2$. $(lo + hi) / 2$ overflows for large indices — a famous real-world bug in the JDK's own binary search.

Q8. The Best Sales Streak (Maximum Subarray — Kadane's)

EASY • Topic: Arrays · DP (Kadane's) • Asked by: Microsoft, LinkedIn (reported 2022–24)

Problem story

A shop logs daily profit or loss as an integer array. The owner wants the most profitable streak of consecutive days. Return the maximum possible sum of a contiguous, non-empty subarray.

Example

Input : nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
 Output: 6 # streak [4, -1, 2, 1]

How to approach (think before you code)

- **Think "best streak ending today".** Either extend yesterday's best streak or start fresh at today: $cur = \max(x, cur + x)$. Track the global best alongside.
- **Why drop a negative prefix?** A running sum below zero only drags down whatever follows — restarting can never hurt. That is the entire proof of Kadane's.
- **Initialise from data.** Start $best = cur = \text{nums}[0]$, never 0 — otherwise an all-negative array (answer should be the largest element) returns 0. Classic trap.

Complexity: Time $O(n)$ · Space $O(1)$

Python

```

def max_subarray(nums):
    best = cur = nums[0]
    for x in nums[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best

```

Java

```
public int maxSubArray(int[] nums) {
    int best = nums[0], cur = nums[0];
    for (int i = 1; i < nums.length; i++) {
        cur = Math.max(nums[i], cur + nums[i]);
        best = Math.max(best, cur);
    }
    return best;
}
```

💡 **Extra tip:** Follow-up they love: "return the streak itself, not just the sum" — track a start index whenever you restart, and save (start, i) whenever best updates.

Q9. The Hostel Staircase (Climbing Stairs)

EASY • Topic: Dynamic Programming (intro) • Asked by: Adobe, Amazon (reported 2022–24)

📖 Problem story

A hostel staircase has n steps. A student climbs 1 or 2 steps at a time. In how many distinct ways can the student reach the top?

Example

Input : $n = 4$
Output: 5 # 1111, 112, 121, 211, 22

How to approach (think before you code)

- **Last-move thinking.** The final move onto step n came from step $n-1$ or $n-2$, so $\text{ways}(n) = \text{ways}(n-1) + \text{ways}(n-2)$ — the Fibonacci recurrence.
- **Why naive recursion dies.** It recomputes the same subproblems exponentially ($O(2^n)$). Overlapping subproblems + optimal substructure = DP.
- **Roll the array.** Each state needs only the previous two — two variables give $O(1)$ space. This "rolling" trick generalises to many DP problems.

Complexity: Time $O(n)$ • Space $O(1)$

Python

```
def climb(n):
    a, b = 1, 1          # ways(0), ways(1)
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b
```

Java

```
public int climbStairs(int n) {
    int a = 1, b = 1;
    for (int i = 2; i <= n; i++) {
        int c = a + b;
        a = b; b = c;
    }
    return b;
}
```

💡 **Extra tip:** Say the three DP words in the interview — "state, transition, base case" — then name them for this problem. That vocabulary alone separates you from most candidates.

Q10. The Missing Attendance Entry (Missing Number)

EASY • Topic: Bit Manipulation / Math • Asked by: Microsoft, Amazon (reported 2022–24)

Problem story

An attendance sheet should contain every roll number from 0 to n , but exactly one entry got smudged. Given the remaining n numbers in any order, find the missing roll number — in one pass and $O(1)$ space.

Example

Input : `nums = [3, 0, 1]`

Output: 2

How to approach (think before you code)

- **Sum formula.** Expected total is $n(n+1)/2$; subtract the actual sum. Simple — but watch overflow for large n in Java (use long).
- **XOR (the elegant one).** $x \oplus x = 0$ and $x \oplus 0 = x$. XOR all indices $0..n$ and all values — every present number cancels, only the missing one survives. No overflow ever.
- **Pattern to remember.** XOR-cancellation also solves "every element appears twice except one" and "two missing numbers" — a whole family of questions.

Complexity: Time $O(n)$ · Space $O(1)$

Python

```
def missing_number(nums):
    ans = len(nums)          # start with n
    for i, x in enumerate(nums):
        ans ^= i ^ x
    return ans
```

Java

```
public int missingNumber(int[] nums) {
    int ans = nums.length;
    for (int i = 0; i < nums.length; i++)
        ans ^= i ^ nums[i];
    return ans;
}
```

 **Extra tip:** When an interviewer says " $O(1)$ space, no modification allowed", think XOR or math before anything else — those constraints are the hint.

MEDIUM

Q11–Q25 · The placement battleground: sliding window, two pointers, DP, trees, graphs, heaps, backtracking. Most coding-round questions live here.

Q11. The Password Screen (Longest Substring Without Repeating Characters)

MEDIUM • Topic: Sliding Window · Hash Map • Asked by: Amazon, Adobe (reported 2022–24) **Problem story**

A login screen shows a scrolling string of characters. A "clean stretch" is a contiguous block with no repeated character.

Given the string, return the length of the longest clean stretch.

Example

Input : s = "abcabcbb"
Output: 3 # "abc"

How to approach (think before you code)

- **Window with a memory.** Grow the right end; remember each character's last index. When the entering character already lies inside the window, jump left just past its previous position.
- **Why $O(n)$, not $O(n^2)$.** Left and right pointers only move forward — each character enters and leaves the window at most once.
- **Validity is monotonic.** Shrinking a duplicate-free window keeps it duplicate-free — that property is what licenses sliding window (playbook §3).

Complexity: Time $O(n)$ · Space $O(k)$ (charset size)

Python

```
def longest_unique(s):
    last = {}
    left = best = 0
    for right, ch in enumerate(s):
        if ch in last and last[ch] >= left:
            left = last[ch] + 1
        last[ch] = right
        best = max(best, right - left + 1)
    return best
```

Java

```
public int lengthOfLongestSubstring(String s) {
    Map<Character, Integer> last = new HashMap<>();
    int left = 0, best = 0;
    for (int right = 0; right < s.length(); right++) {
        char ch = s.charAt(right);
        if (last.containsKey(ch) && last.get(ch) >= left)
            left = last.get(ch) + 1;
        last.put(ch, right);
        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

 **Extra tip:** Dry-run "abba" aloud — it exposes the classic bug of moving left backwards. The guard `last[ch] >= left` exists exactly for this case.

Q12. Three Adjustments That Cancel Out (3Sum)

MEDIUM • Topic: Sorting · Two Pointers • Asked by: Adobe, Meta (reported 2022–24)

Problem story

An accountant's ledger has n signed adjustments. Auditors want every unique triplet of entries that sums to exactly zero — no duplicate triplets in the report.

Example

Input : `nums = [-1, 0, 1, 2, -1, -4]`

Output: `[[-1, -1, 2], [-1, 0, 1]]`

How to approach (think before you code)

- **Sort first.** Sorting enables two things: the two-pointer sweep, and easy duplicate skipping (equal values sit together).
- **Fix one, two-point the rest.** For each index i , find pairs summing to $-\text{nums}[i]$ in the sorted right part: pointers from both ends, move left if the sum is small, right if large.
- **Skip duplicates at all three positions.** After using a value at i , left, or right, skip repeats of it — this is where most wrong answers come from.

Complexity: Time $O(n^2)$ · Space $O(1)$ (ignoring output)

Python

```
def three_sum(nums):
    nums.sort()
    res = []
    for i in range(len(nums) - 2):
        if i > 0 and nums[i] == nums[i - 1]:
            continue
        l, r = i + 1, len(nums) - 1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if s < 0: l += 1
            elif s > 0: r -= 1
            else:
                res.append([nums[i], nums[l], nums[r]])
                l += 1
                while l < r and nums[l] == nums[l - 1]:
                    l += 1
        return res
```

Java

```
public List<List<Integer>> threeSum(int[] nums) {
    Arrays.sort(nums);
    List<List<Integer>> res = new ArrayList<>();
    for (int i = 0; i < nums.length - 2; i++) {
        if (i > 0 && nums[i] == nums[i - 1]) continue;
        int l = i + 1, r = nums.length - 1;
        while (l < r) {
            int s = nums[i] + nums[l] + nums[r];
            if (s < 0) l++;
            else if (s > 0) r--;
            else {
                res.add(Arrays.asList(nums[i], nums[l], nums[r]));
                l++;
                while (l < r && nums[l] == nums[l - 1]) l++;
            }
        }
    }
    return res;
}
```

💡 **Extra tip:** State the target complexity before coding: " $n \leq 3000 \rightarrow O(n^2)$ is intended; $O(n^3)$ brute force won't pass." Interviewers reward that constraint-reading reflex.

Q13. Factory Output Without Your Own Machine (Product of Array Except Self)

MEDIUM • Topic: Arrays · Prefix / Suffix • Asked by: Meta, Amazon (reported 2022–24)

Problem story

A factory floor has n machines; machine i produced $\text{nums}[i]$ units. Each supervisor wants total output of all machines EXCEPT their own — but the division key on the calculator is broken (division not allowed), and it must run in $O(n)$.

Example

Input : $\text{nums} = [1, 2, 3, 4]$
Output: $[24, 12, 8, 6]$

How to approach (think before you code)

- **Answer[i] = (product of left side) × (product of right side).** Two sweeps: a left-to-right pass filling prefix products, then a right-to-left pass multiplying suffix products in.
- **$O(1)$ extra space trick.** Write prefixes into the answer array itself, and carry the suffix as a single running variable in the second pass.
- **Why no division?** Besides the ban, division breaks when any element is 0 — the prefix/suffix method handles zeros for free.

Complexity: Time $O(n)$ · Space $O(1)$ (output not counted)

Python

```
def product_except_self(nums):
    n = len(nums)
    res = [1] * n
    for i in range(1, n):
        res[i] = res[i - 1] * nums[i - 1]
    suffix = 1
    for i in range(n - 1, -1, -1):
        res[i] *= suffix
        suffix *= nums[i]
    return res
```

Java

```
public int[] productExceptSelf(int[] nums) {
    int n = nums.length;
    int[] res = new int[n];
    res[0] = 1;
    for (int i = 1; i < n; i++)
        res[i] = res[i - 1] * nums[i - 1];
    int suffix = 1;
    for (int i = n - 1; i >= 0; i--) {
        res[i] *= suffix;
        suffix *= nums[i];
    }
    return res;
}
```

 **Extra tip:** Prefix/suffix sweeps are a mini-pattern of their own — they also crack "leaders in an array", "trapping rain water" (Q26), and range-product queries.

Q14. The Mirrored Name Tag (Longest Palindromic Substring)

MEDIUM • Topic: Strings · Expand Around Center • Asked by: Amazon, Microsoft (reported 2022–24)

Problem story

A signage shop engraves name tags and offers a discount on the longest part of the name that reads the same forwards and backwards.

Given the string, return its longest palindromic substring.

Example

Input : s = "babad"

Output: "bab" # "aba" also accepted

How to approach (think before you code)

- **Every palindrome has a center.** There are only $2n-1$ centers (each character, and each gap between characters for even lengths). Expand outward from each while the ends match.
- **Why this beats DP here.** Both are $O(n^2)$ time, but expand-around-center is $O(1)$ space and 15 lines; the DP table costs $O(n^2)$ memory. Simpler wins when complexity ties.
- **Handle odd AND even.** Expanding only from single characters misses "abba" — the most common wrong submission on this problem.

Complexity: Time $O(n^2)$ · Space $O(1)$

Python

```
def longest_palindrome(s):
    start = end = 0
    def expand(l, r):
        while l >= 0 and r < len(s) and s[l] == s[r]:
            l -= 1; r += 1
        return l + 1, r - 1
    for i in range(len(s)):
        for a, b in (expand(i, i), expand(i, i + 1)):
            if b - a > end - start:
                start, end = a, b
    return s[start:end + 1]
```

Java

```
public String longestPalindrome(String s) {
    int start = 0, end = 0;
    for (int i = 0; i < s.length(); i++) {
        int a = expand(s, i, i), b = expand(s, i, i + 1);
        int len = Math.max(a, b);
        if (len > end - start + 1) {
            start = i - (len - 1) / 2;
            end = i + len / 2;
        }
    }
    return s.substring(start, end + 1);
}

private int expand(String s, int l, int r) {
    while (l >= 0 && r < s.length()
        && s.charAt(l) == s.charAt(r)) { l--; r++; }
    return r - l - 1;
}
```



```
return r - 1 - 1;    // length of palindrome
}
```

💡 **Extra tip:** If asked "can you do better than $O(n^2)$?", name Manacher's algorithm ($O(n)$) — knowing it exists is enough; coding it is rarely expected in placements.

Q15. The Vending Machine's Change (Coin Change)

MEDIUM • Topic: Dynamic Programming • Asked by: Goldman Sachs, Amazon (reported 2022–24)

Problem story

A vending machine holds unlimited coins of given denominations. For a required change amount, dispense the FEWEST coins possible; if the amount cannot be formed, show -1.

Example

Input : coins = [1, 3, 4], amount = 6

Output: 2 # 3 + 3 (greedy 4+1+1 = 3 coins is wrong!)

How to approach (think before you code)

- **Greedy fails — prove it to yourself.** Largest-coin-first gives 4+1+1 here. Choices affect future value → DP (playbook §3 litmus test).
- **State & transition.** $dp[a]$ = fewest coins to form amount a ; $dp[a] = 1 + \min(dp[a - c])$ over usable coins c . Base: $dp[0] = 0$.
- **Impossible amounts.** Initialise with infinity; if $dp[\text{amount}]$ is still infinity at the end, return -1.

Complexity: Time $O(\text{amount} \times \text{coins})$ • Space $O(\text{amount})$

Python

```
def coin_change(coins, amount):
    INF = float("inf")
    dp = [0] + [INF] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a and dp[a - c] + 1 < dp[a]:
                dp[a] = dp[a - c] + 1
    return dp[amount] if dp[amount] != INF else -1
```

Java

```
public int coinChange(int[] coins, int amount) {
    int INF = amount + 1;           // safe "infinity"
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, INF);
    dp[0] = 0;
    for (int a = 1; a <= amount; a++)
        for (int c : coins)
            if (c <= a)
                dp[a] = Math.min(dp[a], dp[a - c] + 1);
    return dp[amount] == INF ? -1 : dp[amount];
}
```

💡 **Extra tip:** The counting twin — "how many WAYS to make the amount" — swaps min for sum and flips the loop order. Practise both; interviewers pivot between them mid-round.

Q16. The Growing Salary Records (Longest Increasing Subsequence)

MEDIUM • Topic: DP · Binary Search • Asked by: Microsoft, Google (reported 2022–24)

Problem story

An HR system stores an employee's salary after each yearly revision. Analysts want the longest run of records (not necessarily consecutive years) where salary strictly increased.

Example

Input : nums = [10, 9, 2, 5, 3, 7, 101, 18]
Output: 4 # 2, 3, 7, 18 (or 2, 5, 7, 101)

How to approach (think before you code)

- **$O(n^2)$ DP first.** $dp[i]$ = length of the best increasing subsequence ending at $i = 1 + \max(dp[j])$ for $j < i$ with $nums[j] < nums[i]$. Always state this baseline.
- **$O(n \log n)$ upgrade — tails array.** $tails[k]$ = smallest possible tail of an increasing subsequence of length $k+1$. For each x , binary-search the first tail $\geq x$ and replace it (or append). $tails$ stays sorted $\rightarrow$ binary search applies.
- **Careful:** the $tails$ array's LENGTH is the answer; its contents are NOT an actual subsequence.

Complexity: Time $O(n \log n)$ · Space $O(n)$

Python

```
import bisect

def length_of_lis(nums):
    tails = []
    for x in nums:
        i = bisect.bisect_left(tails, x)
        if i == len(tails):
            tails.append(x)
        else:
            tails[i] = x
    return len(tails)
```

Java

```
public int lengthOfLIS(int[] nums) {
    List<Integer> tails = new ArrayList<>();
    for (int x : nums) {
        int i = Collections.binarySearch(tails, x);
        if (i < 0) i = -(i + 1); // insertion point
        if (i == tails.size()) tails.add(x);
        else tails.set(i, x);
    }
    return tails.size();
}
```

 **Extra tip:** "Longest NON-decreasing" changes `bisect_left` to `bisect_right` (allow equal). One word in the statement flips one function call — read carefully.

Q17. Counting Islands on the Satellite Map (Number of Islands)

MEDIUM • Topic: Grid · DFS / BFS • Asked by: Amazon, Google, Bloomberg (reported 2022–24)

Problem story

A satellite photo is a grid of "1" (land) and "0" (water). An island is a group of land cells connected horizontally or vertically. Count the islands.

Example

Input :

```
[["1","1","0"],
             ["0","1","0"],
             ["0","0","1"]]
```

Output: 2

How to approach (think before you code)

- **Grid = implicit graph.** Cells are nodes; 4-directional adjacency is the edges. "Connected groups" → count connected components.
- **Flood fill.** Scan every cell; on unvisited land, increment the count and DFS/BFS to sink the whole island (mark visited by flipping to "0" or using a visited set).
- **DFS recursion depth warning.** A 300×300 all-land grid recurses 90,000 deep — fine in Java, risky in Python. Know the iterative/BFS version too.

Complexity: Time $O(\text{rows} \times \text{cols})$ · Space $O(\text{rows} \times \text{cols})$ worst case

Python

```
def num_islands(grid):
    if not grid: return 0
    R, C = len(grid), len(grid[0])
    def sink(r, c):
        if 0 <= r < R and 0 <= c < C and grid[r][c] == "1":
            grid[r][c] = "0"
            sink(r+1, c); sink(r-1, c)
            sink(r, c+1); sink(r, c-1)
    count = 0
    for r in range(R):
        for c in range(C):
            if grid[r][c] == "1":
                count += 1
                sink(r, c)
    return count
```

Java

```
public int numIslands(char[][] grid) {
    int count = 0;
    for (int r = 0; r < grid.length; r++)
        for (int c = 0; c < grid[0].length; c++)
            if (grid[r][c] == '1') {
                count++;
                sink(grid, r, c);
            }
    return count;
}

private void sink(char[][] g, int r, int c) {
    if (r < 0 || r >= g.length || c < 0 ||
        c >= g[0].length || g[r][c] != '1') return;
    g[r][c] = '0';
    sink(g, r+1, c); sink(g, r-1, c);
    sink(g, r, c+1); sink(g, r, c-1);
}
```

 **Extra tip:** Variants swap the traversal, not the idea: diagonal connectivity (8 directions), "max area of island", "islands II with additions" (Union-Find). Same skeleton every time.

Q18. Ordering the Course Prerequisites (Course Schedule)

MEDIUM • Topic: Graphs · Topological Sort • Asked by: Google, Uber (reported 2022-24)

Problem story

A university lists n courses; some pairs say "take course b before course a ". A student asks: is it even possible to finish all courses, or do the prerequisites form an impossible loop?

Example

Input : $n = 2$, prerequisites = $[[1,0]]$ → Output: true

Input : $n = 2$, prerequisites = $[[1,0],[0,1]]$ → Output: false

How to approach (think before you code)

- **Model as a directed graph.** Edge $b \rightarrow a$ means "b before a". All courses are finishable $\Leftrightarrow$ the graph has no cycle.
- **Kahn's algorithm (BFS).** Compute in-degrees; repeatedly take a course with in-degree 0, "complete" it, and decrement its neighbours. If you complete all n , no cycle exists.
- **Why in-degree 0?** Such a course has no pending prerequisite — it is always safe to take next. A cycle never exposes an in-degree-0 node, so its members are never processed.

Complexity: Time $O(V + E)$ · Space $O(V + E)$

Python

```

from collections import deque

def can_finish(n, prereqs):
    adj = [[] for _ in range(n)]
    indeg = [0] * n
    for a, b in prereqs:
        adj[b].append(a)
        indeg[a] += 1
    q = deque(i for i in range(n) if indeg[i] == 0)
    done = 0
    while q:
        u = q.popleft()
        done += 1
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    return done == n

```

Java

```

public boolean canFinish(int n, int[][] prereqs) {
    List<List<Integer>> adj = new ArrayList<>();
    for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
    int[] indeg = new int[n];
    for (int[] p : prereqs) {
        adj.get(p[1]).add(p[0]);
        indeg[p[0]]++;
    }
    Deque<Integer> q = new ArrayDeque<>();
    for (int i = 0; i < n; i++)
        if (indeg[i] == 0) q.add(i);
    int done = 0;
    while (!q.isEmpty()) {
        int u = q.poll(); done++;
        for (int v : adj.get(u))

```

```

        if (--indeg[v] == 0) q.add(v);
    }
    return done == n;
}

```

Extra tip: Course Schedule II ("return one valid order") is the same code — just collect nodes as you poll them. Any "dependency / build order" story is topological sort in costume.

Q19. The Spoiling Crates (Rotting Oranges)

MEDIUM • Topic: Multi-source BFS • Asked by: Amazon (reported 2022–24)

Problem story

A warehouse grid holds fresh crates (1), spoiled crates (2), and empty slots (0). Every minute, a spoiled crate spoils its 4-directional fresh neighbours.

Return the minutes until no fresh crate remains, or -1 if some crate can never be reached.

Example

Input : `[[2,1,1],[1,1,0],[0,1,1]]`

Output: 4

How to approach (think before you code)

- **Simultaneous spread = level-by-level = BFS.** All spoiled crates act at once each minute, so start BFS with ALL of them in the queue at time 0 (multi-source BFS). Levels = minutes.
- **Track fresh count.** Count fresh crates at the start; decrement on each spoil. If any remain after BFS, return -1 — cleaner than rescanning the grid.
- **Why DFS is wrong here.** DFS follows one path deep, so its "time" is path length, not simultaneous rings. Any "minimum time/steps to spread or reach" story means BFS.

Complexity: Time $O(\text{rows} \times \text{cols})$ • Space $O(\text{rows} \times \text{cols})$

Python

```

from collections import deque

def oranges_rotting(grid):
    R, C = len(grid), len(grid[0])
    q, fresh = deque(), 0
    for r in range(R):
        for c in range(C):
            if grid[r][c] == 2: q.append((r, c))
            elif grid[r][c] == 1: fresh += 1
    minutes = 0
    while q and fresh:
        for _ in range(len(q)):          # one level = one minute
            r, c = q.popleft()
            for dr, dc in ((1,0), (-1,0), (0,1), (0,-1)):
                nr, nc = r + dr, c + dc
                if 0 <= nr < R and 0 <= nc < C and grid[nr][nc] == 1:
                    grid[nr][nc] = 2
                    fresh -= 1
                    q.append((nr, nc))
        minutes += 1
    return -1 if fresh else minutes

```

Java

```
public int orangesRotting(int[][] grid) {
    int R = grid.length, C = grid[0].length, fresh = 0;
    Deque<int[]> q = new ArrayDeque<>();
    for (int r = 0; r < R; r++)
        for (int c = 0; c < C; c++) {
            if (grid[r][c] == 2) q.add(new int[]{r, c});
            else if (grid[r][c] == 1) fresh++;
        }
    int minutes = 0;
    int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};
    while (!q.isEmpty() && fresh > 0) {
        for (int k = q.size(); k > 0; k--) {
            int[] cur = q.poll();
            for (int[] d : dirs) {
                int nr = cur[0] + d[0], nc = cur[1] + d[1];
                if (nr >= 0 && nr < R && nc >= 0 && nc < C
                    && grid[nr][nc] == 1) {
                    grid[nr][nc] = 2; fresh--;
                    q.add(new int[]{nr, nc});
                }
            }
        }
        minutes++;
    }
    return fresh == 0 ? minutes : -1;
}
```

 **Extra tip:** The "for _ in range(len(q))" inner loop is THE multi-level BFS idiom — it freezes one level before the next begins. It reappears in Word Ladder (Q30) and every shortest-steps grid problem.

Q20. The Trending Hashtags (Top K Frequent Elements)

MEDIUM • Topic: Hash Map · Heap / Bucket Sort • Asked by: Meta, Amazon (reported 2022–24)

Problem story

A social platform logs every hashtag used today. The trends panel must show the k most frequently used hashtags — faster than fully sorting the log.

Example

Input : nums = [1,1,1,2,2,3], k = 2

Output: [1, 2]

How to approach (think before you code)

- **Count first.** One hash-map pass gives frequency of every element — the raw material for any "most frequent" question.
- **Heap of size k .** Push (freq, item) into a min-heap capped at k ; the heap always holds the current top k . $O(n \log k)$, better than full sort when $k \ll n$.
- **Bucket sort for $O(n)$.** Frequency is at most n , so make buckets indexed by frequency and read from the top. Mention this upgrade — it is the "wow" answer.

Complexity: Time $O(n)$ bucket / $O(n \log k)$ heap • Space $O(n)$

Python

```
from collections import Counter

def top_k_frequent(nums, k):
```

```
count = Counter(nums)
buckets = [[] for _ in range(len(nums) + 1)]
for item, freq in count.items():
    buckets[freq].append(item)
res = []
for freq in range(len(buckets) - 1, 0, -1):
    for item in buckets[freq]:
        res.append(item)
        if len(res) == k:
            return res
```

Java

```
public int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> count = new HashMap<>();
    for (int x : nums) count.merge(x, 1, Integer::sum);
    List<List<Integer>> buckets = new ArrayList<>();
    for (int i = 0; i <= nums.length; i++)
        buckets.add(new ArrayList<>());
    for (var e : count.entrySet())
        buckets.get(e.getValue()).add(e.getKey());
    int[] res = new int[k]; int idx = 0;
    for (int f = nums.length; f >= 1 && idx < k; f--)
        for (int item : buckets.get(f)) {
            res[idx++] = item;
            if (idx == k) break;
        }
    return res;
}
```

 **Extra tip:** Interviewers often ask "why not just sort?" Answer with numbers: sort is $O(n \log n)$; heap $O(n \log k)$; bucket $O(n)$ — and explain when each is appropriate. The comparison IS the answer.

Q21. Cleaning Up the Meeting Calendar (Merge Intervals)

MEDIUM • Topic: Sorting · Intervals • Asked by: Google, Meta (reported 2022–24)

Problem story

A meeting-room calendar has overlapping bookings after a sync glitch. Merge every set of overlapping time intervals into one continuous block.

Example

Input : `[[1,3],[2,6],[8,10],[15,18]]`

Output: `[[1,6],[8,10],[15,18]]`

How to approach (think before you code)

- **Sort by start time.** After sorting, any interval can only overlap the one currently being built — overlap checking becomes a single comparison.
- **Extend or emit.** If the next interval starts $\leq$ current end, merge (end = max of ends — max matters for contained intervals like `[1,10],[2,3]`). Otherwise emit and start fresh.
- **Read the touching rule.** Do `[1,4]` and `[4,5]` merge? Usually yes ($\text{start} \leq \text{end}$). Confirm with the interviewer — asking IS a good signal.

Complexity: Time $O(n \log n)$ · Space $O(n)$

Python

```
def merge(intervals):
    intervals.sort(key=lambda x: x[0])
    res = [intervals[0]]
    for start, end in intervals[1:]:
        if start <= res[-1][1]:
            res[-1][1] = max(res[-1][1], end)
        else:
            res.append([start, end])
    return res
```

Java

```
public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
    List<int[]> res = new ArrayList<>();
    int[] cur = intervals[0];
    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] <= cur[1])
            cur[1] = Math.max(cur[1], intervals[i][1]);
        else {
            res.add(cur);
            cur = intervals[i];
        }
    }
    res.add(cur);
    return res.toArray(new int[0][]);
}
```

 **Extra tip:** The interval family — insert interval, meeting rooms (min rooms needed), non-overlapping intervals — all begin with "sort by start (or end)". Sorting is the move; the rest is bookkeeping.

Q22. The Rotated Log File (Search in Rotated Sorted Array)

MEDIUM • Topic: Modified Binary Search • Asked by: Amazon, Microsoft (reported 2022–24)

Problem story

A server's sorted log array was rotated at an unknown position during a restart (e.g., [4,5,6,7,0,1,2]). Find the index of a target entry in $O(\log n)$, or -1.

Example

Input : nums = [4,5,6,7,0,1,2], target = 0

Output: 4

How to approach (think before you code)

- **Key insight: one half is always sorted.** At any mid, either [lo..mid] or [mid..hi] is a normal sorted range — check with one comparison ($\text{nums}[\text{lo}] \leq \text{nums}[\text{mid}]$).
- **Decide using the sorted half.** If the target lies inside the sorted half's range, search there; otherwise search the other half. Ordinary binary search structure survives.
- **Follow-up ladder.** "Find the minimum" (find the rotation point) and "duplicates allowed" (worst case degrades to $O(n)$) — be ready for both.

Complexity: Time $O(\log n)$ • Space $O(1)$

Python

```
def search(nums, target):
```



```

lo, hi = 0, len(nums) - 1
while lo <= hi:
    mid = (lo + hi) // 2
    if nums[mid] == target:
        return mid
    if nums[lo] <= nums[mid]:          # left half sorted
        if nums[lo] <= target < nums[mid]:
            hi = mid - 1
        else:
            lo = mid + 1
    else:                             # right half sorted
        if nums[mid] < target <= nums[hi]:
            lo = mid + 1
        else:
            hi = mid - 1
return -1

```

Java

```

public int search(int[] nums, int target) {
    int lo = 0, hi = nums.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] == target) return mid;
        if (nums[lo] <= nums[mid]) { // left sorted
            if (nums[lo] <= target && target < nums[mid])
                hi = mid - 1;
            else lo = mid + 1;
        } else { // right sorted
            if (nums[mid] < target && target <= nums[hi])
                lo = mid + 1;
            else hi = mid - 1;
        }
    }
    return -1;
}

```

 **Extra tip:** Use `<=` in `nums[lo] <= nums[mid]` — a two-element array makes `lo` equal `mid`, and `<` silently misclassifies the half. Test `[3,1]` with target 1.

Q23. Auditing the Corrupted Index (Validate Binary Search Tree)

MEDIUM • Topic: Trees • DFS with Bounds • Asked by: Amazon, Meta (reported 2022–24)

Problem story

A database index is stored as a binary tree and must satisfy the BST rule: every node in a left subtree is smaller than its ancestor, every node in a right subtree larger. After a crash, verify whether the tree is still a valid BST.

Example

Input : [5,1,7,null,null,6,8] → Output: true
 Input : [5,1,7,null,null,4,8] → Output: false # 4 in right subtree of 5

How to approach (think before you code)

- **The classic trap.** Checking only `child < parent < child` passes the second example above wrongly. The rule constrains a node against ALL ancestors, not just its parent.
- **Pass down (min, max) bounds.** Root allows $(-\infty, +\infty)$. Going left tightens max to the node's value; going right tightens min. Every node must fit its inherited window.

- **Alternative:** in-order traversal of a valid BST is strictly increasing — traverse and compare with the previous value. Same complexity; know both.

Complexity: Time $O(n)$ · Space $O(h)$ (recursion, $h = \text{height}$)

Python

```
def is_valid_bst(root):
    def check(node, low, high):
        if not node:
            return True
        if not (low < node.val < high):
            return False
        return (check(node.left, low, node.val) and
                check(node.right, node.val, high))
    return check(root, float("-inf"), float("inf"))
```

Java

```
public boolean isValidBST(TreeNode root) {
    return check(root, Long.MIN_VALUE, Long.MAX_VALUE);
}
private boolean check(TreeNode n, long low, long high) {
    if (n == null) return true;
    if (n.val <= low || n.val >= high) return false;
    return check(n.left, low, n.val)
        && check(n.right, n.val, high);
}
```

💡 **Extra tip:** In Java, use long bounds — a node holding Integer.MAX_VALUE breaks int bounds. Hidden tests include exactly this case.

Q24. Reading the Org Chart Floor by Floor (Binary Tree Level Order Traversal)

MEDIUM · Topic: Trees · BFS · Asked by: Amazon, Microsoft (reported 2022–24)

Problem story

A company's reporting structure is a binary tree. HR wants the employee list printed level by level — CEO first, then their two reports, and so on — each level as its own list.

Example

Input : [3,9,20,null,null,15,7]
Output: [[3], [9,20], [15,7]]

How to approach (think before you code)

- **Level-by-level = queue.** BFS with a queue visits nodes in ring order. To group rings, freeze the queue size at the start of each level and process exactly that many.
- **Same idiom as Rotting Oranges (Q19).** The "for size of queue" inner loop is the universal level-separator — one idiom, many problems.
- **Variants ride the same code.** Zigzag order (reverse alternate levels), right-side view (last node per level), average of levels — all trivial edits of this skeleton.

Complexity: Time $O(n)$ · Space $O(n)$

Python

```
from collections import deque

def level_order(root):
    if not root: return []
    res, q = [], deque([root])
    while q:
        level = []
        for _ in range(len(q)):
            node = q.popleft()
            level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        res.append(level)
    return res
```

Java

```
public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> res = new ArrayList<>();
    if (root == null) return res;
    Deque<TreeNode> q = new ArrayDeque<>();
    q.add(root);
    while (!q.isEmpty()) {
        List<Integer> level = new ArrayList<>();
        for (int k = q.size(); k > 0; k--) {
            TreeNode n = q.poll();
            level.add(n.val);
            if (n.left != null) q.add(n.left);
            if (n.right != null) q.add(n.right);
        }
        res.add(level);
    }
    return res;
}
```

 **Extra tip:** Know all four tree traversals cold (pre/in/post-order recursively AND iteratively, plus level order). Placements test traversal fluency more than any other tree skill.

Q25. Every Possible Meal Combo (Subsets)

MEDIUM • Topic: Backtracking / Recursion • Asked by: Meta, Amazon (reported 2022–24)

Problem story

A cafe with n distinct menu items wants to print every possible combo a customer could order — including the empty order and the full menu. Generate all subsets.

Example

Input : nums = [1, 2, 3]
Output : [[], [1], [2], [3], [1, 2], [1, 3], [2, 3], [1, 2, 3]]

How to approach (think before you code)

- **Include-or-exclude decision tree.** For each item you either take it or skip it $\rightarrow 2^n$ leaves = 2^n subsets. Recursion walks this tree.
- **The backtracking rhythm.** choose $\rightarrow$ recurse $\rightarrow$ un-choose (pop). The un-choose restores state so sibling branches start clean — forgetting it is THE backtracking bug.

- **Copy on record.** Append a COPY of the current path (`path[:]` / `new ArrayList<>(path)`); appending the live reference makes every recorded subset mutate later.

Complexity: Time $O(n \cdot 2^n)$ · Space $O(n)$ recursion depth

Python

```
def subsets(nums):
    res, path = [], []
    def backtrack(i):
        if i == len(nums):
            res.append(path[:])      # copy!
            return
        path.append(nums[i])        # choose
        backtrack(i + 1)
        path.pop()                  # un-choose
        backtrack(i + 1)            # skip
    backtrack(0)
    return res
```

Java

```
public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> res = new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(), res);
    return res;
}

private void backtrack(int[] nums, int i,
    List<Integer> path, List<List<Integer>> res) {
    if (i == nums.length) {
        res.add(new ArrayList<>(path));
        return;
    }
    path.add(nums[i]);
    backtrack(nums, i + 1, path, res);
    path.remove(path.size() - 1);
    backtrack(nums, i + 1, path, res);
}
```

💡 **Extra tip:** Subsets, Permutations, Combination Sum, Palindrome Partitioning — one template: for each choice { choose; recurse; un-choose }. Learn the template once; only the "choices" differ.

HARD

Q26–Q40 · Product-company differentiators: monotonic structures, hard DP, tries, tree DP, design questions. Solving even a few of these cleanly sets you apart.

Q26. Water on the City Skyline (Trapping Rain Water)

HARD · Topic: Arrays · Two Pointers · Asked by: Amazon, Google (reported 2022–24)

Problem story

After a storm, water collects between buildings of a skyline. Building i has height $h[i]$ and width 1. Return the total units of water trapped between the buildings.

Example

Input : h = [0,1,0,2,1,0,1,3,2,1,2,1]

Output: 6

How to approach (think before you code)

- **Per-column formula.** Water above column $i = \min(\text{max height to its left, max height to its right}) - h[i]$, if positive. That formula is the whole problem.
- **From prefix arrays to two pointers.** Precomputing `leftMax[]` and `rightMax[]` gives $O(n)$ time, $O(n)$ space. The upgrade: walk pointers from both ends — whichever side has the smaller max is fully determined (the other side's max can only be $\geq$), so its water can be settled immediately. $O(1)$ space.
- **Convince yourself of the pointer rule.** If `leftMax < rightMax`, then for the left pointer $\min(\text{leftMax, rightMax}) = \text{leftMax}$ regardless of unseen bars — that is why moving the smaller side is safe.

Complexity: Time $O(n)$ · Space $O(1)$

Python

```
def trap(h):
    l, r = 0, len(h) - 1
    left_max = right_max = water = 0
    while l < r:
        if h[l] < h[r]:
            left_max = max(left_max, h[l])
            water += left_max - h[l]
            l += 1
        else:
            right_max = max(right_max, h[r])
            water += right_max - h[r]
            r -= 1
    return water
```

Java

```
public int trap(int[] h) {
    int l = 0, r = h.length - 1;
    int leftMax = 0, rightMax = 0, water = 0;
    while (l < r) {
        if (h[l] < h[r]) {
            leftMax = Math.max(leftMax, h[l]);
            water += leftMax - h[l];
            l++;
        } else {
            rightMax = Math.max(rightMax, h[r]);
            water += rightMax - h[r];
            r--;
        }
    }
    return water;
}
```

💡 **Extra tip:** Present all three solutions in ascending elegance — brute $O(n^2)$, prefix arrays $O(n)/O(n)$, two pointers $O(n)/O(1)$. The narrated ladder scores more than the final code.

Q27. Median Across Two Sorted Branches (Median of Two Sorted Arrays)

HARD • Topic: Binary Search · Partitioning • Asked by: Amazon, Microsoft, Google (reported 2022–24)

Problem story

Two bank branches each keep their customers' account balances in sorted arrays. Head office wants the median balance across both branches combined — in $O(\log(\min(m, n)))$, without merging.

Example

Input : a = [1, 3], b = [2] → Output: 2.0
 Input : a = [1, 2], b = [3, 4] → Output: 2.5

How to approach (think before you code)

- **Median = a partition.** The median splits the combined data into equal halves with every left element $\leq$ every right element. So search for the partition, not the value.
- **Binary search the smaller array.** Choose i elements from a 's left side; then $j = (m+n+1)/2 - i$ must come from b . Valid when $a[i-1] \leq b[j]$ and $b[j-1] \leq a[i]$; otherwise shift i left or right — a binary search on i .
- **Guard the edges with $\pm\infty$.** When a partition has nothing on one side, treat the missing value as $-\infty$ or $+\infty$ — this collapses all edge cases into the same two comparisons.

Complexity: Time $O(\log \min(m, n))$ · Space $O(1)$

Python

```
def find_median(a, b):
    if len(a) > len(b):
        a, b = b, a
    m, n = len(a), len(b)
    lo, hi = 0, m
    half = (m + n + 1) // 2
    while True:
        i = (lo + hi) // 2
        j = half - i
        aL = a[i-1] if i > 0 else float("-inf")
        aR = a[i] if i < m else float("inf")
        bL = b[j-1] if j > 0 else float("-inf")
        bR = b[j] if j < n else float("inf")
        if aL <= bR and bL <= aR:
            if (m + n) % 2:
                return float(max(aL, bL))
            return (max(aL, bL) + min(aR, bR)) / 2
        if aL > bR:
            hi = i - 1
        else:
            lo = i + 1
```

Java

```
public double findMedianSortedArrays(int[] a, int[] b) {
    if (a.length > b.length) return findMedianSortedArrays(b, a);
    int m = a.length, n = b.length;
    int lo = 0, hi = m, half = (m + n + 1) / 2;
    while (true) {
        int i = (lo + hi) / 2, j = half - i;
        int aL = (i > 0) ? a[i-1] : Integer.MIN_VALUE;
        int aR = (i < m) ? a[i] : Integer.MAX_VALUE;
        int bL = (j > 0) ? b[j-1] : Integer.MIN_VALUE;
        int bR = (j < n) ? b[j] : Integer.MAX_VALUE;
        if (aL <= bR && bL <= aR) {
            if ((m + n) & 1) == 1) return Math.max(aL, bL);
            return (Math.max(aL, bL) + Math.min(aR, bR)) / 2.0;
        }
        if (aL > bR) hi = i - 1;
        else lo = i + 1;
    }
}
```

Extra tip: If the $O(\log)$ version slips away under pressure, state the $O(m+n)$ two-pointer merge-count first and bank partial credit — then attempt the partition method.

Q28. Merging Every Branch's Queue (Merge k Sorted Lists)

HARD • Topic: Linked Lists • Min-Heap • Asked by: Amazon, Meta (reported 2022–24)

Problem story

k bank branches each maintain a sorted linked list of pending requests. Head office must combine all k lists into one sorted list, efficiently even when k is large.

Example

Input : $[[1, 4, 5], [1, 3, 4], [2, 6]]$

Output: $1 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 4 \rightarrow 5 \rightarrow 6$

How to approach (think before you code)

- **Generalise Q4.** With k fronts instead of 2, the repeated question becomes "which of k heads is smallest right now?" — a changing-set minimum $\rightarrow$ min-heap.
- **Heap of k heads.** Push each list's head; pop the smallest, attach it, push its successor. Every node passes through the heap once: $O(N \log k)$.
- **Alternative: divide & conquer.** Pair-merge lists like a knockout tournament — also $O(N \log k)$ with no heap. Offer both; explain why naive one-by-one merging is $O(N \cdot k)$.

Complexity: Time $O(N \log k)$ • Space $O(k)$

Python

```
import heapq

def merge_k_lists(lists):
    heap = [(node.val, i, node)
            for i, node in enumerate(lists) if node]
    heapq.heapify(heap)
    dummy = tail = ListNode(0)
    while heap:
        _, i, node = heapq.heappop(heap)
        tail.next = node
        tail = node
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))
    return dummy.next
```

Java

```
public ListNode mergeKLists(ListNode[] lists) {
    PriorityQueue<ListNode> heap =
        new PriorityQueue<>((x, y) -> x.val - y.val);
    for (ListNode node : lists)
        if (node != null) heap.add(node);
    ListNode dummy = new ListNode(0), tail = dummy;
    while (!heap.isEmpty()) {
        ListNode node = heap.poll();
        tail.next = node;
        tail = node;
        if (node.next != null) heap.add(node.next);
    }
}
```

```
    return dummy.next;
}
```

Extra tip: The Python tuple carries an index i as a tiebreaker — without it, equal values make Python compare `ListNode` objects and crash. A tiny detail that fails live coding rounds.

Q29. The Peak Traffic Monitor (Sliding Window Maximum)

HARD • Topic: Monotonic Deque • Asked by: Amazon, Google (reported 2022–24)

Problem story

A network dashboard shows traffic readings per second. For every window of k consecutive seconds, display the peak reading — the naive per-window scan is too slow for a day of data.

Example

Input : `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3`

Output: `[3,3,5,5,6,7]`

How to approach (think before you code)

- **Find the waste.** Rescanning each window is $O(nk)$. Observation: a reading smaller than a LATER reading can never be a future maximum — it is permanently dominated.
- **Monotonic deque of indices.** Keep indices whose values strictly decrease front-to-back. New element: pop smaller values from the back. Front = current max. Pop the front when it slides out of the window.
- **Amortised $O(n)$.** Each index enters and leaves the deque once — total work is linear despite the inner while-loop.

Complexity: Time $O(n)$ • Space $O(k)$

Python

```
from collections import deque

def max_sliding_window(nums, k):
    dq, res = deque(), []
    for i, x in enumerate(nums):
        while dq and nums[dq[-1]] <= x:
            dq.pop()
        dq.append(i)
        if dq[0] == i - k:
            dq.popleft()
        if i >= k - 1:
            res.append(nums[dq[0]])
    return res
```

Java

```
public int[] maxSlidingWindow(int[] nums, int k) {
    Deque<Integer> dq = new ArrayDeque<>();
    int[] res = new int[nums.length - k + 1];
    for (int i = 0; i < nums.length; i++) {
        while (!dq.isEmpty() && nums[dq.peekLast()] <= nums[i])
            dq.pollLast();
        dq.addLast(i);
        if (dq.peekFirst() == i - k) dq.pollFirst();
        if (i >= k - 1) res[i - k + 1] = nums[dq.peekFirst()];
    }
    return res;
}
```



```
}
```

Extra tip: Store INDICES, never values — indices tell you when the front expires. This deque is the same family as monotonic stacks (Q37); learn them together.

Q30. One Letter at a Time (Word Ladder)

HARD • Topic: Graphs · BFS • Asked by: Amazon, Meta, Lyft (reported 2022–24)

Problem story

A word puzzle: transform *beginWord* into *endWord*, changing exactly one letter per step, where every intermediate word must exist in the given dictionary.

Return the length of the shortest transformation sequence, or 0 if impossible.

Example

Input : `begin="hit", end="cog", dict=["hot","dot","dog","lot","log","cog"]`
 Output: 5 # `hit → hot → dot → dog → cog`

How to approach (think before you code)

- **See the hidden graph.** Words are nodes; an edge joins words differing by one letter. "Shortest sequence" in an unweighted graph → BFS. The insight is the modelling, not the traversal.
- **Generate neighbours cheaply.** For each word, try all 26 letters at each position and check membership in a hash set — $O(26L)$ per word beats comparing against every dictionary word.
- **Mark visited on enqueue.** Remove a word from the set the moment you queue it — marking on dequeue lets the same word enter the queue many times and blows up.

Complexity: Time $O(N \cdot 26 \cdot L)$ · Space $O(N)$

Python

```
from collections import deque

def ladder_length(begin, end, word_list):
    words = set(word_list)
    if end not in words:
        return 0
    q = deque([(begin, 1)])
    while q:
        word, steps = q.popleft()
        if word == end:
            return steps
        for i in range(len(word)):
            for c in "abcdefghijklmnopqrstuvwxyz":
                nxt = word[:i] + c + word[i+1:]
                if nxt in words:
                    words.remove(nxt)      # visited
                    q.append((nxt, steps + 1))
    return 0
```

Java

```
public int ladderLength(String begin, String end,
                        List<String> wordList) {
    Set<String> words = new HashSet<>(wordList);
    if (!words.contains(end)) return 0;
    Deque<String> q = new ArrayDeque<>();
```

```

q.add(begin);
int steps = 1;
while (!q.isEmpty()) {
    for (int k = q.size(); k > 0; k--) {
        String w = q.poll();
        if (w.equals(end)) return steps;
        char[] arr = w.toCharArray();
        for (int i = 0; i < arr.length; i++) {
            char old = arr[i];
            for (char c = 'a'; c <= 'z'; c++) {
                arr[i] = c;
                String nxt = new String(arr);
                if (words.remove(nxt)) q.add(nxt);
            }
            arr[i] = old;
        }
    }
    steps++;
}
return 0;
}

```

 **Extra tip:** Mention bidirectional BFS (expand from both ends, always grow the smaller frontier) — it cuts the search space dramatically and is a strong senior-level signal.

Q31. The Spell-Checker's Distance (Edit Distance)

HARD • Topic: Dynamic Programming (2D) • Asked by: Google, Amazon (reported 2022–24)

Problem story

A spell-checker measures how far a typo is from a dictionary word: the minimum number of single-character insertions, deletions, or replacements to convert word1 into word2.

Example

Input : word1 = "horse", word2 = "ros"
Output: 3 # horse → rorse → rose → ros

How to approach (think before you code)

- **State.** $dp[i][j]$ = min operations to convert the first i chars of word1 into the first j chars of word2. Answer: $dp[m][n]$.
- **Transition.** If last chars match: $dp[i][j] = dp[i-1][j-1]$ (free). Else $1 + \min(\text{delete } dp[i-1][j], \text{insert } dp[i][j-1], \text{replace } dp[i-1][j-1])$.
- **Base cases carry meaning.** $dp[i][0] = i$ (delete everything), $dp[0][j] = j$ (insert everything). Explaining WHY each transition maps to an operation is what interviewers probe.

Complexity: Time $O(m \times n)$ • Space $O(m \times n)$ (row-rollable to $O(n)$)

Python

```

def min_distance(w1, w2):
    m, n = len(w1), len(w2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1): dp[i][0] = i
    for j in range(n + 1): dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if w1[i-1] == w2[j-1]:

```

```

        dp[i][j] = dp[i-1][j-1]
    else:
        dp[i][j] = 1 + min(dp[i-1][j],      # delete
                           dp[i][j-1],      # insert
                           dp[i-1][j-1])    # replace

    return dp[m][n]

```

Java

```

public int minDistance(String w1, String w2) {
    int m = w1.length(), n = w2.length();
    int[][] dp = new int[m + 1][n + 1];
    for (int i = 0; i <= m; i++) dp[i][0] = i;
    for (int j = 0; j <= n; j++) dp[0][j] = j;
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++) {
            if (w1.charAt(i-1) == w2.charAt(j-1))
                dp[i][j] = dp[i-1][j-1];
            else
                dp[i][j] = 1 + Math.min(dp[i-1][j-1],
                                         Math.min(dp[i-1][j], dp[i][j-1]));
        }
    return dp[m][n];
}

```

💡 **Extra tip:** Edit Distance, LCS, and Longest Palindromic Subsequence share one 2D-DP skeleton — master this grid once and three "hard" questions become one.

Q32. The Boggle Board (Word Search II)

HARD • Topic: Trie + Backtracking • Asked by: Amazon, Microsoft (reported 2022–24)

Problem story

A word-hunt game shows a letter grid and a dictionary. Find every dictionary word that can be traced through adjacent cells (up/down/left/right) without reusing a cell within one word.

Example

Input : board = [{"o","a","n"}, {"e","t","a"}, {"i","h","k"}],
 words = ["oath","eat","tank"]
 Output: ["oath","eat"]

How to approach (think before you code)

- **Why a Trie.** Searching each word separately repeats work on shared prefixes ("oath", "oat"...). A trie lets ONE DFS walk the grid and the dictionary simultaneously, pruning the instant a prefix dies.
- **DFS with the trie in hand.** From each cell, descend into the trie child matching the letter; no child → prune the entire branch. A node holding a complete word → record it (then clear it to avoid duplicates).
- **Backtrack the board.** Mark the cell (e.g., "#") before recursing, restore after — the standard choose/un-choose rhythm from Q25 applied to a grid.

Complexity: Time $O(\text{cells} \times 4 \times 3^{(L-1)})$ worst case • Space $O(\text{total word chars})$

Python

```

def find_words(board, words):
    trie = {}
    for w in words:
        # build trie

```

```

node = trie
for ch in w:
    node = node.setdefault(ch, {})
node["$"] = w # word marker
R, C, res = len(board), len(board[0]), []
def dfs(r, c, node):
    ch = board[r][c]
    if ch not in node:
        return
    nxt = node[ch]
    if "$" in nxt:
        res.append(nxt.pop("$")) # record once
        board[r][c] = "#" # choose
        for dr, dc in ((1,0), (-1,0), (0,1), (0,-1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < R and 0 <= nc < C and board[nr][nc] != "#":
                dfs(nr, nc, nxt)
        board[r][c] = ch # un-choose
    for r in range(R):
        for c in range(C):
            dfs(r, c, trie)
return res

```

Java

```

class TrieNode {
    TrieNode[] next = new TrieNode[26];
    String word = null;
}

public List<String> findWords(char[][] board, String[] words) {
    TrieNode root = new TrieNode();
    for (String w : words) { // build trie
        TrieNode node = root;
        for (char ch : w.toCharArray()) {
            int i = ch - 'a';
            if (node.next[i] == null) node.next[i] = new TrieNode();
            node = node.next[i];
        }
        node.word = w;
    }
    List<String> res = new ArrayList<>();
    for (int r = 0; r < board.length; r++)
        for (int c = 0; c < board[0].length; c++)
            dfs(board, r, c, root, res);
    return res;
}

private void dfs(char[][] b, int r, int c,
    TrieNode node, List<String> res) {
    if (r < 0 || r >= b.length || c < 0 || c >= b[0].length
        || b[r][c] == '#') return;
    char ch = b[r][c];
    TrieNode nxt = node.next[ch - 'a'];
    if (nxt == null) return;
    if (nxt.word != null) {
        res.add(nxt.word);
        nxt.word = null; // avoid duplicates
    }
    b[r][c] = '#';
    dfs(b, r+1, c, nxt, res); dfs(b, r-1, c, nxt, res);
    dfs(b, r, c+1, nxt, res); dfs(b, r, c-1, nxt, res);
    b[r][c] = ch;
}

```

 **Extra tip:** If Word Search II feels heavy, first master Word Search I (single word, plain backtracking) and Implement Trie separately — this problem is literally those two glued together.

Q33. Seating the Rival Queens (N-Queens)

HARD • Topic: Backtracking • Asked by: Amazon, Adobe (reported 2022–24)

Problem story

Place n chess queens on an $n \times n$ board so that no two attack each other — no shared row, column, or diagonal. Return all distinct board arrangements.

Example

Input : $n = 4$

Output: 2 solutions, e.g. `[".Q..", "...Q", "Q...", "..Q."]`

How to approach (think before you code)

- **One queen per row — by construction.** Place row by row; then only columns and diagonals need checking, shrinking the search space massively.
- **$O(1)$ conflict checks with three sets.** A cell (r, c) lies on diagonal $r - c$ and anti-diagonal $r + c$. Three hash sets (cols, diag, anti) answer "is this cell attacked?" instantly.
- **Choose → recurse → un-choose.** Add to the three sets, recurse to the next row, remove on the way back. Reaching row n means a full valid arrangement — record it.

Complexity: Time $O(n!)$ with pruning • Space $O(n)$

Python

```
def solve_n_queens(n):
    res, cols, diag, anti = [], set(), set(), set()
    placement = [-1] * n
    def place(r):
        if r == n:
            res.append(["." * c + "Q" + "." * (n - c - 1)
                        for c in placement])
            return
        for c in range(n):
            if c in cols or (r - c) in diag or (r + c) in anti:
                continue
            cols.add(c); diag.add(r - c); anti.add(r + c)
            placement[r] = c
            place(r + 1)
            cols.remove(c); diag.remove(r - c); anti.remove(r + c)
    place(0)
    return res
```

Java

```
public List<List<String>> solveNQueens(int n) {
    List<List<String>> res = new ArrayList<>();
    boolean[] cols = new boolean[n];
    boolean[] diag = new boolean[2 * n];
    boolean[] anti = new boolean[2 * n];
    int[] placement = new int[n];
    place(0, n, cols, diag, anti, placement, res);
    return res;
}

private void place(int r, int n, boolean[] cols,
    boolean[] diag, boolean[] anti,
    int[] placement, List<List<String>> res) {
    if (r == n) {
        List<String> board = new ArrayList<>();
        for (int c : placement) {
```

```

        char[] row = new char[n];
        Arrays.fill(row, '.');
        row[c] = 'Q';
        board.add(new String(row));
    }
    res.add(board);
    return;
}
for (int c = 0; c < n; c++) {
    if (cols[c] || diag[r - c + n] || anti[r + c]) continue;
    cols[c] = diag[r - c + n] = anti[r + c] = true;
    placement[r] = c;
    place(r + 1, n, cols, diag, anti, placement, res);
    cols[c] = diag[r - c + n] = anti[r + c] = false;
}
}

```

💡 **Extra tip:** The diagonal identities ($r - c$ constant, $r + c$ constant) reappear in every diagonal-grid problem — derive them once on paper and they are yours forever.

Q34. Backing Up the Org Tree (Serialize & Deserialize Binary Tree)

HARD • Topic: Trees · Design · DFS • Asked by: Meta, Amazon, LinkedIn (reported 2022–24)

Problem story

A service must back up an arbitrary binary tree to a plain string over the network, and later reconstruct the identical tree from that string. Design `serialize()` and `deserialize()`.

Example

Input tree : [1,2,3,null,null,4,5]
 serialize → "1,2,#,#,3,4,#,#,5,#,#" → deserialize → same tree

How to approach (think before you code)

- **Preorder with explicit nulls.** Plain preorder is ambiguous — many trees share it. Writing a marker (#) for every null child makes the string decode to exactly one tree.
- **Deserialize mirrors serialize.** Consume tokens left to right: a value → make the node, then recursively build left subtree, then right. The recursion structure IS the format.
- **Design-question habits.** State the format before coding, mention the delimiter choice, and note that BFS-with-nulls is an equally valid alternative encoding.

Complexity: Time $O(n)$ both ways · Space $O(n)$

Python

```

def serialize(root):
    parts = []
    def dfs(node):
        if not node:
            parts.append("#")
            return
        parts.append(str(node.val))
        dfs(node.left)
        dfs(node.right)
    dfs(root)
    return ",".join(parts)

def deserialize(data):

```

```
tokens = iter(data.split(","))
def build():
    val = next(tokens)
    if val == "#":
        return None
    node = TreeNode(int(val))
    node.left = build()
    node.right = build()
    return node
return build()
```

Java

```
public String serialize(TreeNode root) {
    StringBuilder sb = new StringBuilder();
    ser(root, sb);
    return sb.toString();
}
private void ser(TreeNode n, StringBuilder sb) {
    if (n == null) { sb.append("#,"); return; }
    sb.append(n.val).append(',');
    ser(n.left, sb);
    ser(n.right, sb);
}
public TreeNode deserialize(String data) {
    Deque<String> tokens =
        new ArrayDeque<>(Arrays.asList(data.split(",")));
    return build(tokens);
}
private TreeNode build(Deque<String> tokens) {
    String val = tokens.poll();
    if (val.equals("#")) return null;
    TreeNode node = new TreeNode(Integer.parseInt(val));
    node.left = build(tokens);
    node.right = build(tokens);
    return node;
}
```

 **Extra tip:** Test with a skewed tree and negative values ("-5" must not break your parser). For a BST specifically, preorder WITHOUT null markers suffices — saying so earns bonus points.

Q35. The Most Profitable Reporting Chain (Binary Tree Maximum Path Sum)

HARD • Topic: Trees • DFS • DP on Trees • Asked by: Meta, Amazon (reported 2022–24)

Problem story

Each employee node in a company tree has a value (profit or loss they contribute). A "chain" is any path between two nodes following parent-child links.

Return the maximum possible sum over all chains — the path may start and end anywhere, and may bend at one node.

Example

Input : [-10, 9, 20, null, null, 15, 7]
Output: 42 # 15 + 20 + 7

How to approach (think before you code)

- **Two quantities per node — the key insight.** (1) the best DOWNWARD gain a node offers its parent (node + max(one child gain, 0)); (2) the best PATH THROUGH the node (node + left gain + right gain), which may bend and therefore cannot be passed upward.
- **Clamp negatives to zero.** A child subtree with negative gain is better skipped — max(gain, 0) encodes "don't take it".
- **Global variable pattern.** The recursion RETURNS the passable gain, while the bent-path candidate updates a global best. This return-one-track-another pattern powers many tree-DP questions (diameter, longest univalue path).

Complexity: Time $O(n)$ · Space $O(h)$

Python

```
def max_path_sum(root):
    best = float("-inf")
    def gain(node):
        nonlocal best
        if not node:
            return 0
        left = max(gain(node.left), 0)
        right = max(gain(node.right), 0)
        best = max(best, node.val + left + right)
        return node.val + max(left, right)
    gain(root)
    return best
```

Java

```
private int best = Integer.MIN_VALUE;
public int maxPathSum(TreeNode root) {
    gain(root);
    return best;
}
private int gain(TreeNode node) {
    if (node == null) return 0;
    int left = Math.max(gain(node.left), 0);
    int right = Math.max(gain(node.right), 0);
    best = Math.max(best, node.val + left + right);
    return node.val + Math.max(left, right);
}
```

💡 **Extra tip:** Initialise best to $-\infty$, never 0 — an all-negative tree must return its largest single node. The same all-negatives trap as Kadane's (Q8); interviewers reuse their traps.

Q36. The Shortest Search Snippet (Minimum Window Substring)

HARD · Topic: Sliding Window · Hash Counting · Asked by: Meta, LinkedIn, Amazon (reported 2022–24)

📖 Problem story

A search engine must display the **SHORTEST** snippet of a document s that contains every character of the query t , including duplicates. Return that minimum window, or "" if none exists.

Example

Input : $s = \text{"ADOBECODEBANC"}, t = \text{"ABC"}$
Output: "BANC"

How to approach (think before you code)

- **Variable window, two phases.** Expand right until the window covers all of t ; then contract left as far as coverage survives, recording the best window before each contraction step.
- **$O(1)$ coverage check.** Keep t 's character needs in a map and a single counter "how many distinct characters are fully satisfied". Comparing whole maps per step would silently cost $O(26n)$ — the counter removes it.
- **Every pointer moves only forward** — total work $O(|s|)$, the same amortised argument as every sliding-window proof.

Complexity: Time $O(|s| + |t|)$ · Space $O(\text{charset})$

Python

```
from collections import Counter

def min_window(s, t):
    if not t: return ""
    need = Counter(t)
    missing = len(t)
    left = start = end = 0
    for right, ch in enumerate(s, 1):
        if need[ch] > 0:
            missing -= 1
            need[ch] -= 1
        if missing == 0:
            # window covers t
            while need[s[left]] < 0: # drop surplus
                need[s[left]] += 1
                left += 1
            if end == 0 or right - left < end - start:
                start, end = left, right
            need[s[left]] += 1 # break coverage,
            missing += 1      # move on
            left += 1
    return s[start:end]
```

Java

```
public String minWindow(String s, String t) {
    int[] need = new int[128];
    for (char c : t.toCharArray()) need[c]++;
    int missing = t.length();
    int left = 0, start = 0, end = 0;
    for (int right = 1; right <= s.length(); right++) {
        char ch = s.charAt(right - 1);
        if (need[ch] > 0) missing--;
        need[ch]--;
        if (missing == 0) {
            while (need[s.charAt(left)] < 0)
                need[s.charAt(left++)]++;
            if (end == 0 || right - left < end - start) {
                start = left; end = right;
            }
            need[s.charAt(left++)]++;
            missing++;
        }
    }
    return s.substring(start, end);
}
```

 **Extra tip:** This is the hardest of the sliding-window family. Climb the ladder: Q11 (longest unique) → "at most k distinct" → this. Each adds one moving part to the same frame.

Q37. The Widest Billboard (Largest Rectangle in Histogram)

HARD • Topic: Monotonic Stack • Asked by: Amazon, Google (reported 2022–24)

Problem story

A city skyline is a histogram of building heights (width 1 each). An advertiser wants the largest rectangular billboard that fits entirely within the skyline.

Return that maximum rectangular area.

Example

Input : heights = [2,1,5,6,2,3]

Output: 10 # 5×2 across bars of heights 5 and 6

How to approach (think before you code)

- **Per-bar reframe.** For each bar, the best rectangle using its FULL height extends to the nearest shorter bar on each side. So the real question is: nearest-smaller-to-left and nearest-smaller-to-right for every bar.
- **Monotonic stack finds both in one pass.** Keep indices of increasing heights. When a lower bar arrives, every taller bar on the stack just found its right boundary; its left boundary is the element beneath it on the stack. Pop and compute area.
- **Sentinel trick.** Append a 0-height bar at the end to flush the stack — no special end-of-array code.

Complexity: Time $O(n)$ · Space $O(n)$

Python

```
def largest_rectangle(heights):
    stack = [] # indices, heights increasing
    best = 0
    for i, h in enumerate(heights + [0]): # sentinel
        while stack and heights[stack[-1]] >= h:
            height = heights[stack.pop()]
            left = stack[-1] if stack else -1
            best = max(best, height * (i - left - 1))
        stack.append(i)
    return best
```

Java

```
public int largestRectangleArea(int[] heights) {
    Deque<Integer> stack = new ArrayDeque<>();
    int best = 0, n = heights.length;
    for (int i = 0; i <= n; i++) {
        int h = (i == n) ? 0 : heights[i]; // sentinel
        while (!stack.isEmpty()
            && heights[stack.peek()] >= h) {
            int height = heights[stack.pop()];
            int left = stack.isEmpty() ? -1 : stack.peek();
            best = Math.max(best, height * (i - left - 1));
        }
        stack.push(i);
    }
    return best;
}
```

 **Extra tip:** "Maximal Rectangle in a binary matrix" is this problem run once per row (each row as a histogram of consecutive 1s). One hard problem unlocks another — mention the connection.

Q38. The Live Median Dashboard (Find Median from Data Stream)

HARD • Topic: Two Heaps · Design • Asked by: Amazon, Google (reported 2022–24)

Problem story

A monitoring dashboard receives latency numbers one at a time, endlessly. At any moment it must report the median of everything seen so far — re-sorting on every insert is far too slow.

Example

```
addNum(1), addNum(2) → findMedian() = 1.5
addNum(3)           → findMedian() = 2
```

How to approach (think before you code)

- **Split the stream at the middle.** Keep the smaller half in a MAX-heap and the larger half in a MIN-heap. The median lives at the two heap tops — readable in $O(1)$.
- **Rebalance every insert.** Push onto one heap, move its top to the other, and shift back if sizes differ by more than 1. This guarantees every element lands in the correct half.
- **Invariant to state aloud:** $\max(\text{small half}) \leq \min(\text{large half})$, and sizes differ by at most one. Naming invariants is exactly what design-round interviewers listen for.

Complexity: $\text{addNum } O(\log n)$ · $\text{findMedian } O(1)$ · $\text{Space } O(n)$

Python

```
import heapq

class MedianFinder:
    def __init__(self):
        self.small = [] # max-heap (store negatives)
        self.large = [] # min-heap

    def add_num(self, num):
        heapq.heappush(self.small, -num)
        heapq.heappush(self.large, -heapq.heappop(self.small))
        if len(self.large) > len(self.small):
            heapq.heappush(self.small, -heapq.heappop(self.large))

    def find_median(self):
        if len(self.small) > len(self.large):
            return float(-self.small[0])
        return (-self.small[0] + self.large[0]) / 2
```

Java

```
class MedianFinder {
    private PriorityQueue<Integer> small = // max-heap
        new PriorityQueue<>(Collections.reverseOrder());
    private PriorityQueue<Integer> large = // min-heap
        new PriorityQueue<>();

    public void addNum(int num) {
        small.add(num);
        large.add(small.poll());
        if (large.size() > small.size())
            small.add(large.poll());
    }

    public double findMedian() {
        if (small.size() > large.size()) return small.peek();
        return (small.peek() + large.peek()) / 2.0;
    }
}
```

```
}
}
```

Extra tip: Python has no max-heap — negate values on the way in and out. Follow-up to expect: "what if 99% of numbers are in [0,100]?" → counting buckets beat heaps. Read the constraints again!

Q39. Decoding the Alien Dictionary (Alien Dictionary)

HARD • Topic: Graphs · Topological Sort • Asked by: Google, Airbnb (reported 2022–24)

Problem story

Archaeologists find a dictionary of an alien language — the words are sorted by the aliens' unknown alphabet. Recover any valid ordering of their letters, or return "" if the dictionary is self-contradictory.

Example

Input : words = ["wrt", "wrf", "er", "ett", "rftt"]
Output: "wertf"

How to approach (think before you code)

- **Extract the constraints.** For each adjacent word pair, the FIRST differing character gives one rule: earlier letter < later letter → a directed edge. Identical prefixes beyond that yield nothing.
- **Invalid input trap.** If a longer word precedes its own prefix (["abc", "ab"]), the dictionary is invalid → return "" immediately. Most failed submissions miss exactly this.
- **Topological sort the letter graph.** Kahn's algorithm (Q18's exact engine). If the output covers fewer letters than exist, there is a cycle → contradiction → "".

Complexity: Time $O(\text{total chars})$ · Space $O(1)$ (≤ 26 nodes)

Python

```
from collections import deque

def alien_order(words):
    adj = {c: set() for w in words for c in w}
    indeg = {c: 0 for c in adj}
    for w1, w2 in zip(words, words[1:]):
        for a, b in zip(w1, w2):
            if a != b:
                if b not in adj[a]:
                    adj[a].add(b)
                    indeg[b] += 1
                break
        else:
            if len(w1) > len(w2):
                return "" # no difference found
            # invalid: prefix rule
    q = deque(c for c in indeg if indeg[c] == 0)
    order = []
    while q:
        c = q.popleft()
        order.append(c)
        for nxt in adj[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
                q.append(nxt)
    return "".join(order) if len(order) == len(adj) else ""
```

Java

```
public String alienOrder(String[] words) {
    Map<Character, Set<Character>> adj = new HashMap<>();
    Map<Character, Integer> indeg = new HashMap<>();
    for (String w : words)
        for (char c : w.toCharArray()) {
            adj.putIfAbsent(c, new HashSet<>());
            indeg.putIfAbsent(c, 0);
        }
    for (int i = 0; i < words.length - 1; i++) {
        String w1 = words[i], w2 = words[i + 1];
        if (w1.length() > w2.length()
            && w1.startsWith(w2)) return "";
        int len = Math.min(w1.length(), w2.length());
        for (int j = 0; j < len; j++) {
            char a = w1.charAt(j), b = w2.charAt(j);
            if (a != b) {
                if (adj.get(a).add(b))
                    indeg.merge(b, 1, Integer::sum);
                break;
            }
        }
    }
    Deque<Character> q = new ArrayDeque<>();
    for (var e : indeg.entrySet())
        if (e.getValue() == 0) q.add(e.getKey());
    StringBuilder sb = new StringBuilder();
    while (!q.isEmpty()) {
        char c = q.poll();
        sb.append(c);
        for (char nxt : adj.get(c))
            if (indeg.merge(nxt, -1, Integer::sum) == 0)
                q.add(nxt);
    }
    return sb.length() == adj.size() ? sb.toString() : "";
}
```

 **Extra tip:** This is Course Schedule (Q18) wearing a costume — the hard part is building the graph from the story, not sorting it. Practise EXTRACTING graphs from statements; that skill transfers everywhere.

Q40. The Browser's Memory (LRU Cache)

HARD • Topic: Design • Hash Map + Doubly Linked List • Asked by: Amazon, Adobe, Microsoft (reported 2022–24)

Problem story

A browser caches at most capacity pages. *get(key)* returns a cached page (*-1* if absent); *put(key, value)* inserts or updates one. When full, the *LEAST RECENTLY USED* page is evicted.

Both operations must run in $O(1)$.

Example

```
cap=2: put(1,1) put(2,2) get(1)=1 put(3,3)    # evicts key 2
get(2) = -1
```

How to approach (think before you code)

- **Two structures, two jobs.** $O(1)$ lookup $\rightarrow$ hash map. $O(1)$ reorder-by-recency and $O(1)$ evict-oldest $\rightarrow$ doubly linked list (most-recent at head, LRU at tail). The map stores key $\rightarrow$ list-node so both connect.

- **Every operation = unlink + relink.** get: move the node to the head. put existing: update + move to head. put new when full: remove the tail node AND its map entry, then insert at head.
- **Sentinel head & tail nodes** eliminate every null check during unlinking — the same dummy-node trick as Q4, doubled.

Complexity: get $O(1)$ · put $O(1)$ · Space $O(\text{capacity})$

Python

```
class Node:
    def __init__(self, k=0, v=0):
        self.k, self.v = k, v
        self.prev = self.next = None

class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.map = {}
        self.head, self.tail = Node(), Node() # sentinels
        self.head.next = self.tail
        self.tail.prev = self.head

    def _unlink(self, n):
        n.prev.next, n.next.prev = n.next, n.prev

    def _to_front(self, n):
        n.next = self.head.next
        n.prev = self.head
        self.head.next.prev = n
        self.head.next = n

    def get(self, key):
        if key not in self.map:
            return -1
        n = self.map[key]
        self._unlink(n)
        self._to_front(n)
        return n.v

    def put(self, key, value):
        if key in self.map:
            self._unlink(self.map[key])
        n = Node(key, value)
        self.map[key] = n
        self._to_front(n)
        if len(self.map) > self.cap:
            lru = self.tail.prev
            self._unlink(lru)
            del self.map[lru.k]
```

Java

```
class LRUCache {
    class Node {
        int k, v; Node prev, next;
        Node(int k, int v) { this.k = k; this.v = v; }
    }
    private final int cap;
    private final Map<Integer, Node> map = new HashMap<>();
    private final Node head = new Node(0, 0);
    private final Node tail = new Node(0, 0);

    public LRUCache(int capacity) {
        cap = capacity;
        head.next = tail; tail.prev = head;
    }

    private void unlink(Node n) {
        n.prev.next = n.next;
        n.next.prev = n.prev;
    }

    private void moveToHead(Node n) {
        n.next = head.next;
        head.next.prev = n;
        head.next = n;
        n.prev = head;
    }

    public int get(int key) {
        if (!map.containsKey(key)) return -1;
        Node n = map.get(key);
        unlink(n);
        moveToHead(n);
        return n.v;
    }

    public void put(int key, int value) {
        if (map.containsKey(key)) {
            Node n = map.get(key);
            n.v = value;
            unlink(n);
            moveToHead(n);
        } else {
            Node n = new Node(key, value);
            map.put(key, n);
            moveToHead(n);
            if (map.size() > cap) {
                Node lru = tail.prev;
                unlink(lru);
                map.remove(lru.k);
            }
        }
    }
}
```

```

    }
    private void unlink(Node n) {
        n.prev.next = n.next; n.next.prev = n.prev;
    }
    private void toFront(Node n) {
        n.next = head.next; n.prev = head;
        head.next.prev = n; head.next = n;
    }
    public int get(int key) {
        Node n = map.get(key);
        if (n == null) return -1;
        unlink(n); toFront(n);
        return n.v;
    }
    public void put(int key, int value) {
        Node old = map.get(key);
        if (old != null) unlink(old);
        Node n = new Node(key, value);
        map.put(key, n); toFront(n);
        if (map.size() > cap) {
            Node lru = tail.prev;
            unlink(lru);
            map.remove(lru.k);
        }
    }
}

```

 **Extra tip:** Java's LinkedHashMap can solve this in 10 lines — mention it, but implement the manual version: interviewers ask this question precisely to watch you build the doubly linked list. Store the KEY inside each node; eviction needs it to clean the map.

After the 40 — your revision plan

Week 1–2: all Easy + Q11–Q18. **Week 3–4:** remaining Medium + Q26–Q30. **Week 5–6:** remaining Hard.
Week 7: re-solve every question you failed, from a blank editor.

The pass bar: for any question in this bank, you can (a) name the pattern in 60 seconds, (b) explain the approach aloud, and (c) code it bug-free in your drive language within 25 minutes. That bar, met 40 times, IS placement readiness for DSA.